

```
In [2]: # Video Compression - Homework 3 - Motion Estimation
# Le Minh Bang - 黎明朋
# 313540015
```

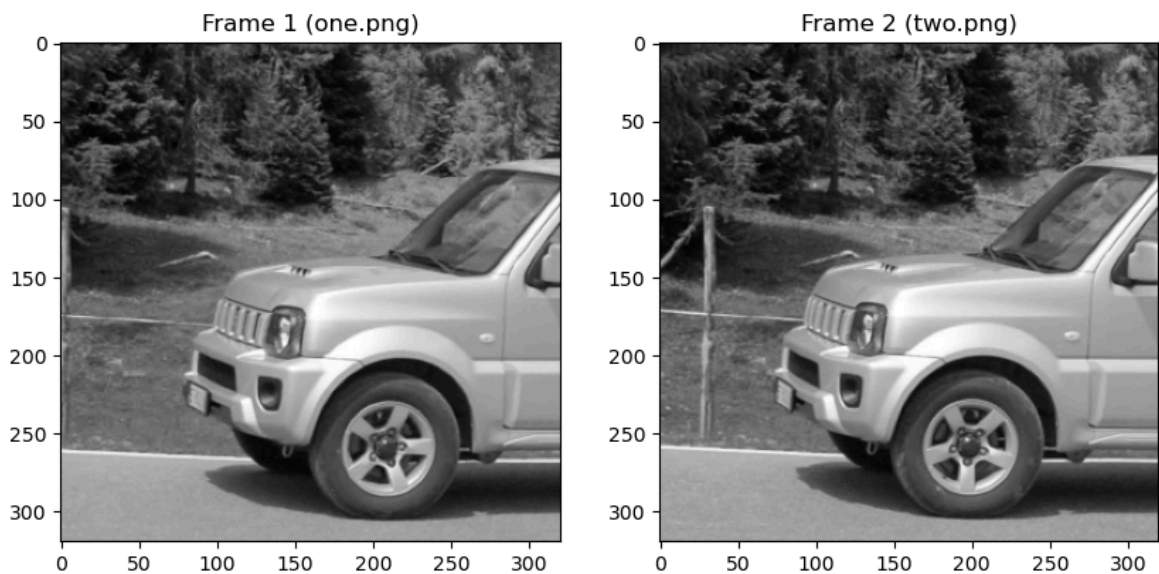
```
In [36]: # Step 1: import and Load frame
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math

frame1 = cv2.imread("one_gray.png", cv2.IMREAD_GRAYSCALE)
frame2 = cv2.imread("two_gray.png", cv2.IMREAD_GRAYSCALE)

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(frame1, cmap="gray")
plt.title("Frame 1 (one.png)")

plt.subplot(1,2,2)
plt.imshow(frame2, cmap="gray")
plt.title("Frame 2 (two.png)")
```

```
Out[36]: Text(0.5, 1.0, 'Frame 2 (two.png)')
```



```
In [37]: # Step 2: Divide the image into blocks
```

```
block_size = 8
search_range = 8
height, width = frame1.shape

num_blocks_y = height // block_size
num_blocks_x = width // block_size
```

```
In [38]: # Step 3: Define functions
```

```
def calculate_psnr(original, predicted):
    mse = np.mean((original - predicted) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel_value = 255.0
```

```

psnr = 20 * math.log10(max_pixel_value / math.sqrt(mse))
return psnr

def compute_sad(block1, block2):
    return np.mean(np.abs(block1 - block2))

def full_search_motion_estimation(
    num_blocks_x,
    num_blocks_y,
    search_range,
    block_size,
    height,
    width
):
    motion_vectors = np.zeros((num_blocks_x, num_blocks_y, 2))
    for y in range(num_blocks_y):
        for x in range(num_blocks_x):
            # Extract the block from the current frame (frame 2)
            block_current = frame2[y*block_size:(y+1)*block_size, x*block_size:(x+1)*block_size]

            # Initialize variables to track the best match
            min_sad = float('inf') # Positive infinity to find the smaller
            best_match = (0, 0)

            # Search within the search range in the reference frame (frame 1)
            for dy in range(-search_range, search_range+1):
                for dx in range(-search_range, search_range+1):
                    ref_y = y*block_size+dy
                    ref_x = x*block_size+dx
                    if ref_y < 0 or ref_y + block_size > height or ref_x < 0 or ref_x + block_size > width:
                        continue

                    # Extract the block from the references frame
                    block_reference = frame1[ref_y:ref_y+block_size, ref_x:ref_x+block_size]

                    # Compute SAD between the current block and candidate reference
                    sad = compute_sad(block_current, block_reference)

                    # Update the best match if this candidate block has a lower SAD
                    if sad < min_sad:
                        min_sad = sad
                        best_match = (dy, dx)

            # Store the motion vector for this block
            motion_vectors[y, x] = best_match
    return motion_vectors

def create_predicted_frame(
    num_blocks_x,
    num_blocks_y,
    motion_vectors,
    block_size,
    height,
    width
):
    predicted_frame = np.zeros_like(frame2)

    for y in range(num_blocks_y):
        for x in range(num_blocks_x):

```

```

        # Get the motion vector for this block
        dy, dx = motion_vectors[y,x]

        # Calculate the position of the corresponding block in frame1
        ref_y = int(y*block_size + dy)
        ref_x = int(x*block_size + dx)

        # Ensure the reference block is within bounds
        if ref_y >= 0 and ref_y + block_size <= height and ref_x >= 0 and re
            # Copy the reference block to the predicted frame
            predicted_frame[y*block_size: (y+1)*block_size, x*block_size: (x

    return predicted_frame

def create_residual_frame(original_frame, predicted_frame):
    residual_frame = original_frame - predicted_frame
    return residual_frame

```

```

In [39]: # Step 4: Create motion vectors, predicted, and residual frame
motion_vectors = full_search_motion_estimation(
    num_blocks_x,
    num_blocks_y,
    search_range,
    block_size,
    height,
    width
)

predicted_frame = create_predicted_frame(
    num_blocks_x,
    num_blocks_y,
    motion_vectors,
    block_size,
    height,
    width
)

residual_frame = create_residual_frame(
    frame2,
    predicted_frame
)

```

```

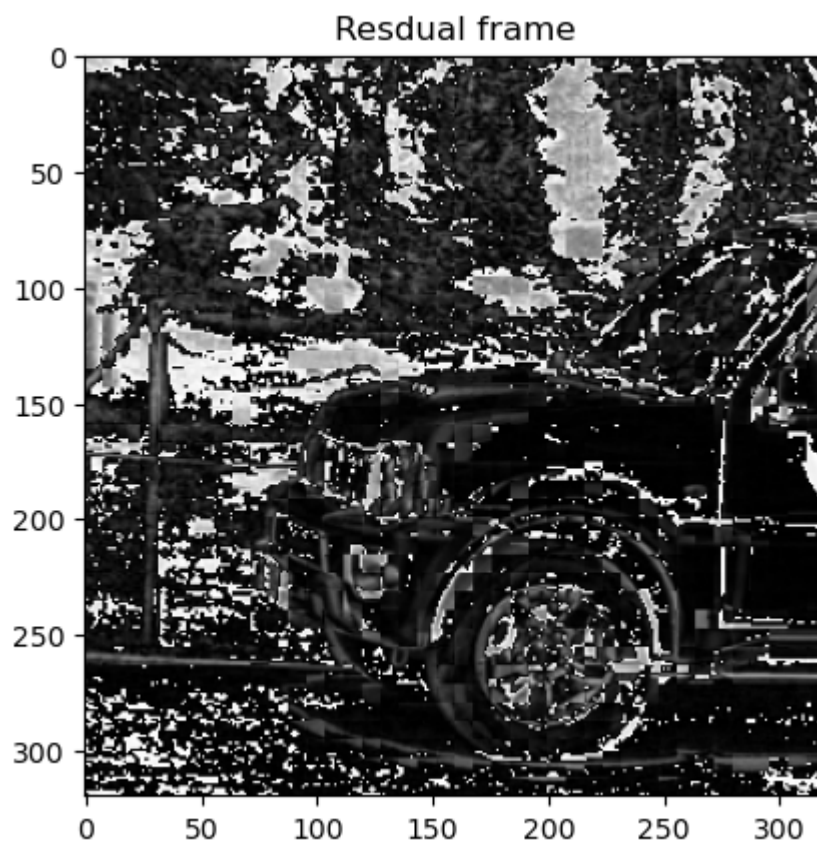
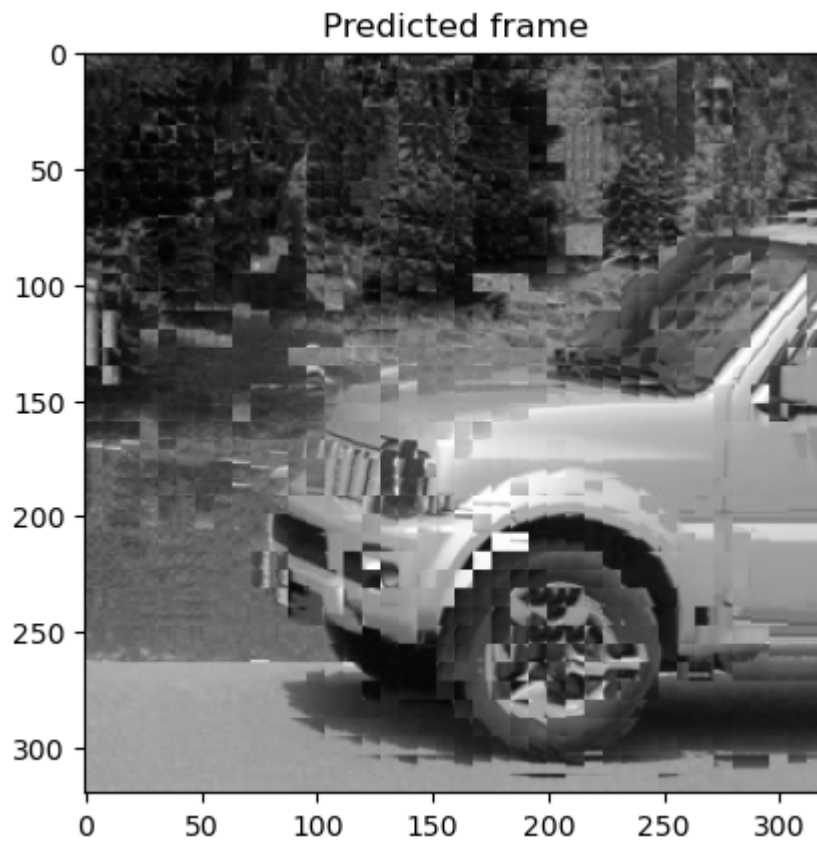
In [40]: # Step 5: Display images, PSNR

# Display predicted frame
plt.imshow(predicted_frame, cmap="gray")
plt.title("Predicted frame")
plt.show()

# Display residual frame
plt.imshow(residual_frame, cmap="gray")
plt.title("Residual frame")
plt.show()

# Show PSNR
psnr = calculate_psnr(frame2, predicted_frame)
print(f"PSNR between actual frame and predicted frame: {psnr} dB")

```



PSNR between actual frame and predicted frame: 29.222998931416736 dB